

Core Question: Machine learning papers rely heavily on *NFE* (*Number of Function Evaluations*) as the sole measure of Neural ODE cost. **Our goal is to test:** Does NFE actually predict real wall-clock runtime across different model sizes, numerical solvers, and stiff regimes, or do other factors (solver loop overhead, Jacobians, hardware synchronization) dominate?

How the 15 Papers Were Organized & Why (3 Groups)

Instead of treating all papers the same, we split them into 3 distinct functional groups to systematically isolate *architecture choices* from *solver/hardware overheads*:

Group A: Conventional MLPs

6 Papers

Purpose: Evidence to freeze one standard baseline architecture.

- **Kim et al. (2021)** — Stiff Neural ODEs (Chaos)
- **Finlay et al. (2020)** — Regularization & NFE (ICML)
- **Caldana & Hesthaven (2025)** — Stiff Systems ROM (IJNME)
- **Chen et al. (2018)** — Original Neural ODE (NeurIPS)
- **Ghosh et al. (2020)** — STEER temporal regularization
- **Onken & Ruthotto (2020)** — Discretize vs Optimize

Group B: Modified Architectures

3 Papers

Purpose: See how modifying dynamics affects per-step cost vs NFE.

- **Dupont et al. (2019)** — Augmented Neural ODEs (adds extra dimensions to simplify flow and lower NFE)
- **Xia et al. (2021)** — Heavy Ball Neural ODE (2nd-order momentum formulation)
- **Poli et al. (2020)** — Hypersolvers (learned auxiliary solvers for inference)

Group C: Solvers & Profiling

6 Papers

Purpose: Solver implementations, loop time, and GPU mechanics.

- **Lienen & Günnemann (2022)** — torchode (GPU loop time)
- **Rackauckas et al. (2019)** — DiffEqFlux.jl (Julia GPU compilation)
- **Kidger (2022)** — Oxford PhD Thesis (diffraX/JAX)
- **Zhuang et al. (2020)** — ACA (Checkpointing adjoint)
- **Zhuang et al. (2021)** — MALI (Reversible integrator)
- **Rackauckas et al. (2020)** — Universal DEs (SciML)

Proof of Work: Archive Status

All **15 original PDFs** were downloaded directly from official arXiv / journal sources and organized into `literature/Group_A_MLP_Neural_ODE/`, `Group_B_Modified_Neural_ODE/`, and `Group_C_Computational_Methodology/`. Every file was validated via byte-header inspection (all valid PDFs, zero missing).

Selection Rationale: We did NOT pick papers based on fame or citations. We selected the 5 papers that directly answer: (1) *What architecture to lock*, (2) *How to separate solver overhead from model runtime*, and (3) *What happens under stiffness*.

Paper & Venue	Group	Why Chosen / Idea Behind Selection	Extracted Detail / Metric	What We Adapt
Kim et al. (2021) <i>Chaos (AIP)</i>	Group A	Stiffness reality-check: Shows explicit solvers (Dopri5) fail or explode on stiff systems. Proves implicit solvers need Jacobians and LU factorizations that <i>NFE completely ignores</i> .	MLP (1–6 layers, 50 nodes, GELU/Tanh). Evaluated on ROBER & POLLU chemical kinetics.	Stiff vs non-stiff comparison; failure threshold tracking.
Finlay et al. (2020) <i>ICML 2020</i>	Group A	NFE vs wall-clock coupling: Measures actual wall-clock hours alongside NFE. Proves that on a fixed model, regularizing flow reduces NFE and runtime simultaneously.	MLP/CNN (depth 3–6, width 64, Softplus). Solved with Dopri5 ($\text{atol}/\text{rtol}=1\text{e-}5$) on 4x RTX 2080 Ti GPUs.	Baseline test to check if runtime strictly tracks NFE when width is fixed.
Lienen & Günnemann (2022) <i>NeurIPS 2022 W.</i>	Group C	Hardware & solver profiling: Decomposes step time into $\text{Model Time } (T_f) + \text{Loop Time } (LT)$. Shows standard batching causes up to 4x step inflation due to sample coupling.	Profiles Dopri5 & Tsit5 on GTX 1080 Ti GPU + Xeon CPU. Tracks per-sample steps, rejected steps, loop latency.	Precise timing breakdown formula: $\text{Step Time} = T_f + T_{\text{solver}}$.
Caldana & Hesthaven (2025) <i>IJNME Journal</i>	Group A	Concrete stiff benchmark: Formalizes Van der Pol oscillator across stiffness ($\mu \in [1, 100]$). Measures $\# \text{fev}$ (NFE), $\# \text{jev}$ (Jacobians), $\# \text{lu}$ (decompositions), and CPU seconds.	Dense FNN (3–10 layers, 30–100 width, GELU/SiLU). Evaluates explicit RK45 vs implicit Radau IIA / BDF.	Van der Pol benchmark setup & formal metrics ($\# \text{fev}$, $\# \text{jev}$, $\# \text{lu}$, runtime).
Dupont et al. (2019) <i>NeurIPS 2019</i>	Group B	Decoupling state size from NFE: Shows adding extra zero dimensions simplifies flow and drops NFE from >100 to <30 , while slightly increasing per-step forward cost.	MLP (1–3 layers, 16–64 width, ReLU). State augmented with $p \in \{1, 2, 5, 10\}$ dimensions.	Controlled test: vary state dimension d vs hidden width H .

Basis for Architecture Lock (Phase 4)

Based on the overlap across Finlay (width 64), Dupont (width 64), and Kim (50 nodes), we lock **exactly one standard architecture** to avoid confounding model search with performance:
Input $y(t) \in \mathbb{R}^d \rightarrow \text{Linear}(d+1 \rightarrow 64) \rightarrow \text{GELU} \rightarrow \text{Linear}(64 \rightarrow 64) \rightarrow \text{GELU} \rightarrow \text{Linear}(64 \rightarrow d)$
• **~4,500 parameters (for d=2)**. Smooth GELU ensures valid Jacobians for implicit solvers.

Next Immediate Experimental Steps

- **Exp 1 (NFE vs Runtime):** Fixed model, vary tolerance ϵ does wall-clock track NFE linearly?
- **Exp 2 (Capacity Effect):** Vary width $H \in \{16, 32, 64, 128, 256\}$ at fixed NFE ϵ measure T_f impact.
- **Exp 3 (Stiffness Sweep):** Van der Pol ($\mu = 1$ to 100) ϵ observe where explicit solvers explode.